LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA

JOBSHEET III: ARRAY OF OBJECTS

Nama : Faren Hafiza Afanda

NIM : 254107020025

Kelas : Teknik Informatika – 1C

3.1 Tujuan Praktikum

Setelah nyobain praktikum ini, mahasiswa diharapkan bisa:

1. Paham fungsi dari *array* yang isinya adalah objek.
2. Bisa mempraktikkan cara pembuatan (*instansiasi*) *array of objects* di program Java.
3. Bisa mengolah dan ngutak-ngatik data yang ada di dalam *array of objects* tersebut.

3.2 & 3.3 Membuat Array dari Object dan Menerima Input Menggunakan Looping

(Catatan: Keseluruhan kode program untuk percobaan ini bisa dilihat pada file Mahasiswa15.java dan MahasiswaDemo.java di dalam repository GitHub).

A. Hasil Output Program

```
Masukkan Data Mahasiswa ke-1
NIM   : 254107020025
Nama  : Faren
Kelas : TI-1C
IPK   : 4
-----------------------------------
Masukkan Data Mahasiswa ke-2
NIM   : 9291029321
Nama  : Hafiza
Kelas : TI-1D
IPK   : 3.9
-----------------------------------
Masukkan Data Mahasiswa ke-3
NIM   : 28281901
Nama  : Afanda
Kelas : TI-1E
IPK   : 3.8
-----------------------------------

=== HASIL DATA MAHASISWA ===
Data Mahasiswa ke-1
NIM   : 254107020025
Nama  : Faren
Kelas : TI-1C
IPK   : 4.0
-----------------------------------
Data Mahasiswa ke-2
NIM   : 9291029321
Nama  : Hafiza
Kelas : TI-1D
IPK   : 3.9
-----------------------------------
Data Mahasiswa ke-3
NIM   : 28281901
Nama  : Afanda
Kelas : TI-1E
IPK   : 3.8
-----------------------------------
```

B. Jawaban Pertanyaan 3.2.3

1. **Berdasarkan uji coba 3.2, apakah class yang akan dibuat array of object harus selalu memiliki atribut dan sekaligus method? Jelaskan!**

   Jawaban: Sebenarnya tidak harus. Sebuah *class* bisa saja cuma dipakai untuk menyimpan data (hanya punya atribut). Tapi, karena kita belajar Pemrograman Berorientasi Objek (OOP), idealnya sebuah *class* itu punya data (atribut) dan juga fungsinya (*method*).

2. **Apa yang dilakukan oleh kode program berikut?** *Mahasiswa[] arrayOfMahasiswa = new Mahasiswa[3];*

   Jawaban: Kode ini fungsinya untuk membuat "rak" atau wadah *array* bernama arrayOfMahasiswa yang berkapasitas 3 slot. Nantinya, ketiga slot ini akan diisi oleh data bertipe objek Mahasiswa.

3. **memiliki Apakah class Mahasiswa konstruktor? Jika tidak, kenapa bisa dilakukan pemanggilan konstruktur pada baris program berikut?** *arrayOfMahasiswa[0] = new Mahasiswa();*

   Jawaban: Di kode aslinya memang tidak kita tulis. Tapi Java itu pintar, dia otomatis membuatkan "konstruktor bawaan" (*default constructor*) yang kosong saat kompilasi. Makanya, kita tetap bisa memanggil new Mahasiswa() tanpa *error*.

4. **Apa yang dilakukan oleh kode program pengisian atribut tersebut?** *Jawaban:* Baris pertama itu ibaratnya membentuk wujud fisik objek Mahasiswa-nya terlebih dahulu. Kemudian, baris-baris di bawahnya digunakan untuk memasukkan data NIM, Nama, Kelas, dan IPK ke dalam objek yang baru saja dibuat tersebut.

5. **Mengapa class Mahasiswa dan MahasiswaDemo dipisahkan pada uji coba 3.2?** *Jawaban:* Biar kodenya rapi dan tugasnya jelas. *Class* Mahasiswa fokus menjadi cetakan datanya saja, sedangkan *class* MahasiswaDemo fokus untuk mengeksekusi dan menjalankan programnya (karena di dalamnya ada fungsi main).

C. Jawaban Pertanyaan 3.3.3

1. **Tambahkan method cetakInfo() pada class Mahasiswa kemudian modifikasi kode program pada langkah no 3.** *Jawaban:* Method cetakInfo() sudah ditambahkan di *class* Mahasiswa agar proses cetak data lebih rapi. Jadi, di file Demo-nya kita tidak perlu menulis System.out.println panjang-panjang, cukup memanggil sintaks arrayOfMahasiswa[i].cetakInfo(); di dalam *looping*.

2. **Misalkan Anda punya array baru bertipe array of Mahasiswa dengan nama myArrayOfMahasiswa. Mengapa kode berikut menyebabkan error?**

   Jawaban: Karena kita baru bikin "rak" kosongnya saja, tapi belum menciptakan fisik objeknya. Harusnya, sebelum mengisi atribut nim, kita wajib membuat objeknya dulu

dengan kode: myArrayOfMahasiswa[0] = new Mahasiswa();. Karena terlewat, programnya bingung dan memunculkan *error NullPointerException*.

3.4 Constructor Berparameter

(Catatan: Keseluruhan kode program untuk percobaan ini bisa dilihat pada file MataKuliah15.java dan MataKuliahDemo.java di dalam repository GitHub).

A. Hasil Output Program

```
Masukkan Data Mata Kuliah ke-1
Kode:
ASD26
Nama:
Algoritma & Struktur Data
SKS:
12
Jumlah Jam:
4
----------------------------------
Masukkan Data Mata Kuliah ke-2
Kode:
BD26
Nama:
Basis Data
SKS:
12
Jumlah Jam:
3
----------------------------------
Masukkan Data Mata Kuliah ke-3
Kode:
AL26
Nama:
Aljabar Linear
SKS:
100
Jumlah Jam:
3
----------------------------------

=== HASIL DATA MAHASISWA ===
Data Mata Kuliah ke-1
Kode: ASD26
Nama: Algoritma & Struktur Data
SKS: 12
Jumlah Jam: 4
----------------------------------
Data Mata Kuliah ke-2
Kode: BD26
```

B. Jawaban Pertanyaan 3.4.3

1. Apakah suatu class dapat memiliki lebih dari 1 constructor? Jika iya, berikan contohnya! *Jawaban:* Bisa banget. Konsep ini dinamakan *Constructor Overloading*. Contohnya, di dalam satu *class*, kita bisa menaruh konstruktor kosong dan konstruktor yang ada parameternya sekaligus secara bersamaan.

2. **Tambahkan method tambahData() pada class Matakuliah, kemudian gunakan method tersebut di class MatakuliahDemo untuk menambahkan data Matakuliah!** *Jawaban:* Karena di praktikum ini kita sudah menggunakan *Constructor* berparameter, proses pengisian data langsung ter- *handle* sekaligus saat pembuatan objek dengan sintaks new MataKuliah15(kode, nama, sks, jumlahJam);.
3. **Tambahkan method cetakInfo() pada class Matakuliah, kemudian gunakan method tersebut di class MatakuliahDemo untuk menampilkan data hasil inputan di layar!** *Jawaban:* Method cetakInfo() sudah disiapkan di *class* MataKuliah15 agar lebih praktis. Di program utama, kita tinggal memanggilnya di dalam *looping* dengan kode arrayOfMatakuliah[i].cetakInfo();.
4. **Modifikasi kode program pada class MatakuliahDemo agar panjang (jumlah elemen) dari array of object Matakuliah ditentukan oleh user melalui input dengan Scanner!** *Jawaban:* Program sudah dimodifikasi. Kita meminta *user* memasukkan angka yang ditampung dalam variabel elemen. Angka inilah yang kemudian dijadikan penentu batas jumlah loker array-nya dengan kode: new MataKuliah15[elemen].

3.5 Tugas (Sistem Informasi Data Dosen)

(Catatan: Keseluruhan kode program untuk penyelesaian tugas ini dapat dilihat pada file Dosen15.java, DataDosen15.java, dan DosenDemo15.java di dalam repository).

A. Hasil Output Program

```
Masukkan jumlah dosen yang ingin di-input
2
Masukkan Data Dosen ke-1
Kode:
291039123
Nama:
Vivi Nur Wijayaningrum, S.Kom., M.Kom.
Jenis Kelamin (Pria/Wanita):
Wanita
Usia:
30
----------------------------------
Masukkan Data Dosen ke-2
Kode:
1231123123
Nama:
Prof. Dr. Eng. Rosa Andrie Asmara, S.T., M.T.
Jenis Kelamin (Pria/Wanita):
Pria
Usia:
40
----------------------------------
=== DATA SEMUA DOSEN ===
Kode: 291039123
Nama: Vivi Nur Wijayaningrum, S.Kom., M.Kom.
Jenis Kelamin: Wanita
Usia: 30
----------------------------------
Kode: 1231123123
Nama: Prof. Dr. Eng. Rosa Andrie Asmara, S.T., M.T.
Jenis Kelamin: Pria
Usia: 40
----------------------------------
JUMLAH DOSEN PER JENIS KELAMIN
Jumlah Pria   : 1
Jumlah Wanita : 1
```

```
=== DOSEN PALING TUA ===
Nama : Prof. Dr. Eng. Rosa Andrie Asmara, S.T., M.T.
Usia : 40

=== DOSEN PALING MUDA ===
Nama : Vivi Nur Wijayaningrum, S.Kom., M.Kom.
Usia : 30
```